

API Automation Challenge Documentation

Autor

Jose Miguel Mateo

Objetivo

Automatizar pruebas funcionales e integración para una API REST utilizando Playwright con TypeScript.

Tecnologías Utilizadas

- Playwright
 - TypeScript
 - Node.js
 - REST API Testing
-

Arquitectura del Proyecto

El proyecto fue desarrollado utilizando buenas prácticas de automatización para garantizar:

- mantenibilidad,
 - reutilización,
 - estabilidad,
 - escalabilidad.
-

Estructura del Proyecto

Banreservas-API-Automation/

```
|
├── tests/
├── docs/
├── screenshots/
├── playwright.config.ts
├── package.json
├── tsconfig.json
├── .env.example
└── README.md
```

Endpoints Automatizados

- POST /Library/Addbook.php
 - GET /Library/GetBook.php
 - POST /Library/DeleteBook.php
-

Flujo Automatizado

La automatización implementa el siguiente flujo CRUD:

1. Crear libros dinámicamente
 2. Consultar libros creados
 3. Eliminar libros
 4. Validar eliminación de recursos
-

Validaciones Implementadas

- Validación de status codes
 - Validación de respuestas JSON
 - Validación de recursos creados
 - Validación de eliminación correcta
 - Validación de recursos inexistentes
-

Buenas Prácticas Implementadas

- Uso de variables dinámicas
 - Uso de variables de entorno
 - Código reutilizable
 - Arquitectura mantenible
 - Assertions robustas
 - Reportes HTML
 - Configuración centralizada
-

Ejecución del Proyecto

Instalar dependencias

`npm install`

Ejecutar pruebas

`npx playwright test`

Abrir reporte

`npx playwright show-report`

Resultado Final

3 pruebas ejecutadas
3 pruebas exitosas
0 fallos

Conclusión

La automatización API fue implementada exitosamente utilizando Playwright y TypeScript aplicando buenas prácticas de automatización, integración y validación de servicios REST.

La solución fue desarrollada considerando mantenibilidad, estabilidad y reutilización.